CYPHER BANK ATM

Technical documentation and code guide

Purpose
Explain how the local Java ATM simulation is structured, how its screens and workflows operate, how data is protected, and what each major method is responsible for.

- Source file: BankingSystem.java
- Runtime: Java 11 or newer; the project was most recently compiled with JDK 27
- Interface: terminal-based ATM kiosk with optional color, sound, and animation
- Persistence: validated local binary account store plus administrator credentials file
- Document revision: 06 November 2026

Important boundary

This is a local learning demonstration. It models account operations but does not connect to a bank, card network, payment processor, or real customer data. It must not be represented as a production banking system.

1. How the program was created

The application was designed as a single-file Java console project so that the presentation layer, transaction rules, file storage, credential protection, and generated sounds are easy to inspect in one place. It uses only standard Java APIs: no web server, database, framework, or external library is required.

Design goals

- Demonstrate a believable ATM-style flow without using real financial infrastructure.

- Store money as integer cents so decimal arithmetic remains exact.

- Persist account data safely with validation and atomic file replacement.

- Avoid storing customer PINs or administrator passwords in readable form.

- Work in ordinary Windows terminals while degrading cleanly when ANSI colors or password masking are unavailable.

Build and start

```
javac BankingSystem.java
java BankingSystem

# Optional operating modes
java BankingSystem --silent
java BankingSystem --no-animation
```

The program reads and writes its data relative to the folder from which it is started. Therefore, it should be launched from the project directory so accounts.bank and admin.properties stay alongside the source.

Runtime lifecycle

```
main()
  -> loadAccounts()
  -> loadAdministrator()
  -> seedAccountsIfEmpty() when safe
  -> welcomeScreen()
  -> mainMenu()
  -> close Scanner on exit
```

This order matters. Existing data is loaded before the interface is shown. Demo accounts are created only when the account store is empty and did not fail validation; a corrupted store is never silently replaced.

2. Project anatomy

Area	What it contains	Why it exists
Configuration	File paths, data limits, PBKDF2 parameters, branding, colors, and feature flags.	Keeps important operational values centralized and readable.
Domain model	Account and AdminCredentials nested classes.	Represents account state and administrator credential material.
Customer workflow	Login, balance, deposit, withdrawal, transfer, statements, PIN change, and account detail screens.	Implements the ATM experience and transaction rules.
Administration	Account search, status controls, PIN reset, admin password change, reporting, and deletion.	Provides local maintenance capabilities behind administrator login.
Persistence and security	Validated binary storage, migration, atomic writes, random salts, and PBKDF2 verification.	Protects local data from accidental corruption and avoids plaintext credentials.
Presentation	Panels, menus, prompts, receipt formatting, progress animation, colors, and tones.	Makes the console feel like an ATM while remaining usable without enhancements.

3. Data model and account rules

Account object

Each Account stores the customer name, account number, salted PIN hash, balance in cents, lock state, and a limited transaction list. Older fields remain only to support a one-time migration from the legacy serialized format; new data does not use Java serialization.

Member or method	Responsibility
name / accountNumber	Identify the account owner and local account reference.
balanceCents	Stores currency as a long integer. For example, KSh 12.50 is stored as 1250.
blocked	Prevents customer login after three failed PIN entries until an administrator restores access.
transactions	Keeps the most recent entries, capped at MAX_TRANSACTIONS.
setPin(newPin)	Generates a fresh random salt and replaces the stored PIN hash.
matchesPin(candidate)	Recomputes a PBKDF2 hash for a supplied PIN and compares it safely.
migrateLegacyFields()	Converts old plaintext PIN and double balance values into the current protected representation.
addTransaction(text)	Adds a timestamped activity record and removes the oldest record when the history limit is reached.

Money and validation

```
toCents(BigDecimal amount)
  -> move decimal point two places
  -> reject more than two decimal places
  -> return exact long cents value

money(long cents)
  -> format the amount as KSh with two decimal places
```

The program deliberately does not use floating-point arithmetic for current balances or transactions. It accepts up to two decimal places and enforces a maximum single transaction to limit unsafe values.

4. Customer journey

Screen or method	What it does	Key checks
welcomeScreen()	Shows bank branding and starts a short secure-session animation.	Presentation only.
mainMenu()	Offers customer access, ATM information, staff administration, and session exit.	Only accepts the displayed range of menu numbers.
customerLogin()	Collects account number and masked PIN, then opens the account menu.	Unknown or blocked accounts are rejected; three failures lock the account.
checkBalance()	Shows account holder, masked reference, and available balance.	Read-only.
deposit()	Accepts a positive amount, shows note-counting feedback, updates balance, saves, and prints a receipt.	Prevents zero, negative, oversized, and overflow values.
withdraw()	Checks available funds, simulates authorization and cash dispense, updates the balance, saves, and prints a receipt.	Rejects an amount above the available balance.
transfer()	Moves money from one active local account to another after confirmation.	Rejects self-transfer, missing or locked recipients, insufficient funds, and recipient overflow.
transactionHistory()	Shows the newest 15 transaction entries.	Shows a notice if no entries exist.
changePin()	Allows a customer to replace their own PIN after entering the existing PIN.	Requires current PIN, valid four digits, and matching confirmation.
accountDetails()	Shows name, account reference, balance, status, and transaction count.	Read-only.

5. Administrator journey

Staff administration is accessed from the main screen. The first use creates a local administrator username and a password of at least 12 characters. Later use requires both the matching username and password.

Method	What the code does
ensureAdministratorConfigured()	Runs the first-time setup only when there is no valid administrator profile and the previous settings did not fail to load.
adminLogin()	Reads a username and password, verifies them against the stored PBKDF2 credential data, clears the temporary password array, and then enters adminMenu().
viewAllAccounts() / searchAccount()	Displays a summary table or matching account records for local maintenance.
bankStatistics()	Counts active and locked accounts and totals all stored customer funds with overflow protection.
toggleBlock()	Flips an account between ACTIVE and LOCKED, records the action, and saves the update.
resetCustomerPin()	Lets an authenticated administrator select a customer, set and confirm a new four-digit PIN, unlock the account, audit the reset, and save it.
changeAdministratorPassword()	Requires the current admin password, requires a confirmed new password, creates a new salt and PBKDF2 hash, and saves the new administrator credentials.
deleteAccount()	Deletes an account only when its balance is exactly zero and the administrator types DELETE.

The two password-management operations have distinct purposes. Customer credentials are short ATM PINs and are reset by an authorized administrator. The administrator password is a longer staff credential and can be changed only by the already authenticated administrator who supplies the current password.

6. Storage, recovery, and security

Files created beside the program

File	Contents	Handling
accounts.bank	Account data, balances, status, credential material, and transaction history in a custom binary format.	Read with bounds and field validation before values are accepted.
admin.properties	Administrator username, salt, and PBKDF2 password hash.	Never stores the readable administrator password.
accounts.dat.migrated.bak	One-time backup of an older legacy account store when migration succeeds.	Keep as recovery evidence until the converted data is verified.

Important persistence methods

Method	What it does
loadAccounts()	Chooses safe binary loading or a restricted one-time legacy import; records failure instead of overwriting unreadable data.
readSafeAccounts()	Validates file magic, format version, record counts, text lengths, credential fields, amounts, and transaction counts before loading.
saveAccounts()	Writes a complete validated account snapshot through writeAtomically().
loadAdministrator() / saveAdministrator()	Read and write the local administrator properties only after expected fields and Base64 sizes are validated.
writeAtomically()	Writes to a temporary file, then moves it into place so a crash is less likely to replace good data with a partial file.
validateAccount()	Checks account fields are safe and within supported limits before they are written.

Credential protection

```
randomBytes(16)
  -> creates a unique salt

hashSecret(secret, salt)
  -> PBKDF2WithHmacSHA256
  -> 150,000 iterations
  -> 32-byte derived value

verifySecret(candidate, salt, expectedHash)
  -> derives candidate hash
  -> uses MessageDigest.isEqual for comparison
```

This protects against casually reading credentials from local files. It is not, by itself, a full banking security model: the application remains a single-user local demonstration without external identity, audit infrastructure, encryption at rest, or operational monitoring.

7. Display, sound, and interaction helpers

Method or constant	Purpose
ANSI / color()	Enables terminal escape colors only when a console is present and NO_COLOR is not set; otherwise strings are plain text.
header(), panelBorder(), panelLine()	Build the fixed-width ATM panel with bank, screen title, terminal ID, location, and instructions.
menuItem(), footer(), line()	Provide a consistent numbered menu system and visual dividers.
animatedInstruction(), animatedPrompt()	Animate text at TYPEWRITER_DELAY_MS, or display it immediately when --no-animation is supplied.
atmSequence()	Renders a progress frame sequence for events such as PIN verification, cash dispense, and session closure.
printReceipt()	Builds a receipt with terminal context, location, date, reference, transaction type, masked account, amount, balance, and status.
maskAccount()	Shows only the last four characters of a local account reference on customer-facing screens and receipts.
transactionReference()	Builds a local timestamp-based reference for receipt traceability.
playSound() / playTonePattern()	Starts optional daemon-thread confirmation, dispense, startup, and error tones. Sound can be disabled with --silent.
clearScreen()	Uses ANSI clear commands when possible, otherwise prints blank lines for older consoles.

The recent kiosk polish deliberately keeps the application terminal-based. It adds bank context, terminal identity, a customer-oriented primary menu, masked references, receipt traceability, and much faster prompt animation without changing the core accounting rules.

8. Input and utility methods

Method	Responsibility
readNonEmpty()	Prompts until the user enters non-whitespace text.
readInt()	Prompts until an integer in the requested menu range is supplied.
readAmount(), readPositiveAmount(), readNonNegativeAmount()	Parse and validate money input before converting it to cents.
readName(), readAdminUsername()	Restrict name and administrator username formats to predictable local values.
readSecret()	Uses Console.readPassword() when supported; otherwise falls back to typed input for IDE consoles.
readPin(), readNewPin(), readNewAdminPassword()	Apply PIN and administrator-password rules and show helpful errors.
findAccount() / generateAccountNumber()	Find a stored account by its reference or generate the next numeric reference for a new account.
success(), error(), warning()	Show semantically distinct messages and select an appropriate optional sound.

9. Operating notes and limitations

Demo account access

On a new installation, the program seeds three demonstration accounts. Their PINs are documented in README.md and should be changed before using the program for any personal demonstration. After an administrator setup, staff credentials should also be treated as private local data.

Safe handling practices

- Do not share accounts.bank, admin.properties, or legacy backup files. They contain sensitive local account data even though passwords are hashed.

- If a data file fails to load, copy it before attempting manual recovery. The application intentionally avoids overwriting a malformed account store with demo data.

- Use --silent when tones are undesirable and --no-animation for a fast non-animated terminal session.

- Use a normal terminal for private PIN and password entry because some IDE consoles cannot mask typed characters.

Production gap

A real banking system needs substantially more: a database with concurrency control, encrypted backups, server-side authorization, an external or managed identity service, immutable audit logs, fraud controls, payment-network integrations, monitoring, incident response, legal and regulatory controls, and independent security review. This program does not provide those capabilities.

Suggested next engineering steps

- Split the single class into UI, account service, credential service, repositories, and audit components.

- Add automated tests for monetary arithmetic, lockouts, transfers, PIN resets, and malformed-data recovery.

- Replace local binary files with an encrypted database or service API for multi-user use.

- Create a separate graphical customer kiosk and staff dashboard if a richer visual experience is needed.

- Add an immutable audit record for each staff credential action, including reset reason and operator identity.

Document conclusion
Cypher Bank ATM is a compact Java learning project that combines a console interface with exact money handling, password hashing, validated persistence, transaction history, and role-specific maintenance features. The code is best understood as an educational prototype with deliberately visible building blocks, not as a real bank system.